IE 643: Deep Learning - Theory and Practice **July-Dec 2026**

Project Report: PINN-based Inversion of Spatially-Varying Diffusivity

Team Name: Binary_Brains *Team Members: 24b2451, 24b2443*

Jash Taparia (24b2451) · Mihir Gune (24b2443)

Code: `https://github.com/bloodrepaer/IE643-Binary-Brains-PINN-Inversion`

Abstract

We want to find the unknown diffusivity $D(x, y)$ of a plate, which changes from place to place, using only a few noisy temperature readings. This is an inverse problem for the heat equation $u_t = \nabla \cdot (D \nabla u)$. We compare two methods on the same synthetic data. The *classical* method solves the heat equation exactly at every step and gets its gradient from an adjoint solve. The *physics-informed neural network (PINN)* learns the temperature and D with two neural networks and adds the heat equation to the loss as a penalty. We built a verified solver and 389 benchmark datasets. At last, A Streamlit app interface that runs both these methods on generated, benchmark or uploaded data from the user

1 Introduction

Many problems need a material property that cannot be measured directly. Examples are the heat conductivity map of a composite plate with hidden defects. What we *can* measure is the temperature (or concentration) $u(x, y, t)$ at a few sensors across the sample. Finding the property from these readings is an *inverse problem*. It is hard for two reasons: many different maps can explain the same few readings, and small noise in the readings can cause large errors in the answer.

The standard way to solve it is classical PDE-constrained optimisation: describe D with a fixed set of numbers, solve the PDE exactly with a numerical solver, and update D using a gradient from an adjoint solve. PINN offers another way. A physics-informed neural network (PINN) uses neural networks to represent the temperature and D, and uses automatic differentiation to put the PDE into the loss. Because the PDE is only a penalty, a PINN can still be trained when part of the physics (for example a boundary condition) is unknown. Whether PINNs are also more accurate is an open question. This project builds both methods on the same data and finds where the PINN does better.

We survey related work in Section 3. Our methods are described in Section 4, and the data in Section 5. We give details on experiments in Section 6 and the results in Section 7. Future work for novelty assessment is in Section 8 , and we conclude in Section 9.

2 Project Workflow

- **Problem definition:** set up the problem in full: the heat equation, the plate, the starting and edge conditions, what we estimate ($\log D$), how we measure error, and six types of test maps for D.
- **Literature review:** PINNs for inverse problems, PINNs for varying diffusivity, and studies comparing the adjoint method with PINNs.

- **Forward solver and datasets:** a finite-volume solver with three time steppers, checked for accuracy; 389 synthetic datasets.
- **Classical baseline:** exact discrete adjoint with a checked gradient; run on all 389 datasets.
- **PINN:** two-network PINN and a loss-weight sweep.
- **Head-to-head experiments:** a 20-dataset study and a 44-dataset sweep, both methods on identical data.
- **Where can a PINN win?** Hypothesis tests with matched controls, a study where both methods use the same network for D, and a dense-sensor sweep.
- **Interface and packaging:** Streamlit app, documented code and saved models.

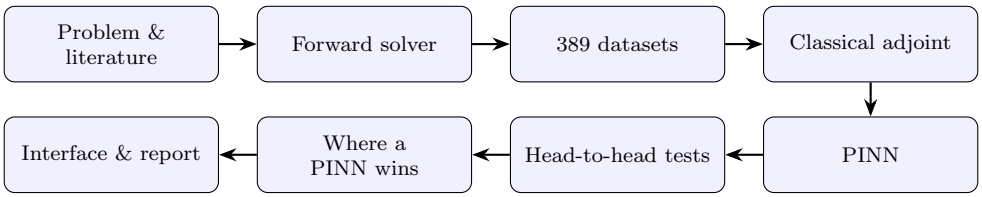


Figure 1: Project workflow.

3 Literature Survey

Physics-informed neural networks. Our understanding of PINNs started with the paper by Raissi et al., which introduced the idea. It showed us that a single neural network can be trained to satisfy both the measured data and the governing PDE. The loss has two parts: the error against the data, and the error in the PDE at extra collocation points, where the derivatives are computed by automatic differentiation. Unknown parameters of the PDE can be trained together with the network, so the same framework solves both forward and inverse problems.

PINNs for spatially varying diffusivity. Thakur et al. is the closest work to ours. They use one network for concentration and one for $D(x, y)$, keep the divergence form $\nabla \cdot (D\nabla c)$, We are following their two- network design

Adjoint method versus PINNs. Zhang, Alla and Karniadakis compare the two approaches on four problems (Burgers, Darcy flow, Allen–Cahn and Navier–Stokes) with the same physics and data. They find that how the unknown is represented largely decides the winner: a grid or a few numbers favour the adjoint, and a neural network favours the PINN. For long time-dependent or high-dimensional problems, storing the adjoint's trajectory becomes expensive and PINNs give good accuracy at lower cost. We use this paper for our matched comparison later in the extended studies post we did the initial exploration D.

4 Proposed Approach or Approaches

Problem. On the unit square $\Omega = [0, 1]^2$ and time $t \in [0, T]$,

$$\frac{\partial u}{\partial t} = \nabla \cdot \big(D(x, y)\nabla u\big) = D\,\Delta u + \nabla D \cdot \nabla u, \qquad u(\cdot, 0) = u_0, \quad u|_{\partial\Omega} = g(t). \tag{1}$$

We get noisy readings $d_i = u(x_i, y_i, t_i) + \varepsilon_i$ with $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ and want $D > 0$. We always solve for $m = \log D$, which keeps D positive. We keep the full divergence form, because the term $\nabla D \cdot \nabla u$ tells us *where* D changes.

4.1 Work done before prep-presentation review

- Defined the problem, reviewed the literature and planned five parts: forward solver, synthetic datasets, classical adjoint baseline, PINN, and a hybrid PINN→adjoint method.The hybrid PINN and adjoint method is now planned in Novelty Assessment
- First forward solver: Crank–Nicolson with a 9-point stencil, with data simulated on the same grid used for inversion. We later moved onto 5-point stencil as it had higher accuracy and also used further forward solvers like RK4, Forward Euler etc. along with Crank Nicholson
- First datasets: absolute noise $\sigma \in \{0.01, 0.05\}$, eight random reading times per file, and seeds from Python's `hash()`.

4.2 Work done after prep-presentation review

Forward solver. We use finite volumes on an $N \times N$ grid. Between two neighbouring nodes we use the harmonic mean $2D_pD_q/(D_p + D_q)$, because the two half-cells act like two resistances in series. This gives $\dot{u} = L(D)u + b(t)$ with L sparse and symmetric. Three time steppers share L: RK4 (to make data), Crank–Nicolson with two backward-Euler half-steps at the start (Rannacher Start), and forward Euler (benchmark only). The Crank–Nicolson matrix is factorised once per D and reused for every forward and adjoint step.

Classical method (discrete adjoint). D is described by 16×16 numbers θ, interpolated to the 65×65 grid, with $10^{-3} \le D \le 10$. We minimise

$$J(\theta) = \frac{1}{2\sigma^2}\big\|Q\,u(m) - d\big\|^2 + \alpha R(m), \qquad R_{\mathrm{Tik}} = \tfrac{1}{2}\int |\nabla m|^2, \quad R_{\mathrm{TV}} = \int \sqrt{|\nabla m|^2 + \epsilon}, \tag{2}$$

where $u(m)$ is the exact Crank–Nicolson solution and Q reads it off at the sensors (which can lie between grid points). One forward and one backward (adjoint) sweep give the gradient for all 256 numbers at once. The below algorithm specifies the given method

Algorithm 1 Classical adjoint inversion

```
Start from the best constant D; set α_0 = N_d (number of readings)
for k = 0, 1, 2, ... with α_k = N_d 10^{-k/2} do
    repeat                                   ▷ L-BFGS-B, at most 150 iterations, warm start
        Forward solve: u(m) by Crank–Nicolson
        Adjoint solve backwards in time; gradient ∂J/∂θ
        L-BFGS-B update of θ within the bounds on D
    until converged
    if RMS misfit ≤ σ then stop
```

PINN architecture. Two networks:

- *Temperature network* $u_\theta(x, y, t)$: random Fourier features (scale 0.35), then three tanh layers of 80 units (19,521 parameters). Its output is a simple analytic guess for the initial heat bumps plus a

learned correction multiplied by $16x(1-x)y(1-y)$ and $\tanh(8t/T)$. So the zero boundary and the initial condition hold exactly.

- *Diffusivity network* $m_\phi(x, y)$: Fourier features, then three tanh layers of 64 units (12,545 parameters). It outputs a correction to a trainable background $\log D$, with $D = \exp(\text{clamp}(m, -5, 2))$. It has no time input, because D is spatially varying and doesn't vary with time

The following table mentions the reasons for the parameters chosen:

Component	Why it is there
Two separate networks	u depends on (x, y, t) but D only on (x, y); keeping them apart builds this in.
Fourier features	Plain networks learn only smooth shapes at first (spectral bias); Fourier features let them fit sharper detail.
tanh activation	The loss needs second derivatives of u; ReLU networks have zero curvature almost everywhere.
Output $\log D$	Keeps D positive and treats relative errors equally.
Exact initial and boundary conditions	Removes two loss terms and their weights, and the network cannot violate them.
Divergence-form residual	The $\nabla D \cdot \nabla u$ term carries the information about where D changes.
Delayed start of D training	If D learns before u fits the data, a wrong u drags D to a wrong answer.

Table 1: PINN components and the reason for each.

PINN loss.

$$L = 40\, L_{\text{data}} + 20\, L_{\text{PDE}} + 10\, L_{\text{BC}} + 20\, L_{\text{IC}} + \lambda R(\log D). \tag{3}$$

L_{data} is the mean squared error at the sensors, divided by the mean squared reading; it is damped once it falls below $0.95\sigma^2$, so the network does not fit the noise. L_{PDE} is the mean squared error of $u_t - (D_x u_x + D_y u_y + D\Delta u)$ at 768 random points per step. It is the only term that gives information about D; 90% of its gradient goes to the D network and 10% to the temperature network, so the PDE mainly shapes D instead of bending u. R is Tikhonov ($\lambda = 2 \times 10^{-4}$) for smooth fields or total variation (3×10^{-4}) for sharp ones. For tests with an unknown boundary, a variant (`pinn_inversion_bc.py`) removes the boundary constraint, so wall values come only from the data and the PDE.

The following algorithm describes the full PINN training procedure.

Algorithm 2 PINN training

1: **for** epoch = 1 to 1500 **do**
2: Sample 768 new collocation points (x, y, t)
3: Compute u_θ and D_ϕ, and get u_t, ∇u, Δu, ∇D by automatic differentiation
4: Compute the loss in Eq. (3)
5: Update θ with AdamW
6: Update ϕ with AdamW, with a learning rate near 0 for about 50 epochs, then ramped up over 15% of training
7: Return the best checkpoint from the last 35% of training

Same network for D in both methods. The grid adjoint and the PINN differ in two ways at once: how D is stored (grid vs network) and how the physics is used (exact vs penalty). To separate the two, both methods also use the same D network (`NeuralDiffusivity`: 3 tanh layers of 32 units in float64, $D \in [10^{-3}, 10]$ through a sigmoid, same starting weights):

- *CN neural adjoint:* the same Crank–Nicolson solver and adjoint give the gradient on the grid, which is passed back through the network and optimised with L-BFGS-B.
- *Neural PINN:* a temperature network trained together with this D network (Adam, then L-BFGS).

For the dense-sensor study we used a *BE neural adjoint* (backward Euler, 100 steps) against a PINN with a matching backward-Euler time loss on 4,096 random time pairs. Both used $\alpha = 1$ and seed 42.

Interface. A Streamlit app (`app/streamlit_app.py`) calls our solvers unchanged. It has four tabs. *Data* lets you generate a dataset, pick one of the 389, or upload a CSV with columns `x,y,t,u[,sigma]`. *Run* holds the settings for both methods. *Results* shows the replay error for both methods, the error against the truth when known, maps, error maps, loss curves and downloads. *Load model* (reload a saved model or one of 40 bundled models). Every data source is saved in the benchmark format, so both solvers read user data exactly like benchmark data

5 Data set Details

Type and source. All data are synthetic, numerical, space–time data made by our own code; nothing was downloaded from any website. One command (`python generate_dataset_v2.py`, about 4 minutes on a laptop CPU) writes the benchmark: 389 datasets and 35 ground-truth simulations, 175 MB in total.

True D fields. Six families plus 11 variants: A smooth sinusoid, B Gaussian bump, C circular inclusion, D layered/checkerboard, E Gaussian random field, F plate with defects.

How a dataset is made.

1. Build the true D on a 129×129 grid.
2. Choose the time horizon from the Fourier number $\mathrm{Fo} \in \{0.02, 0.05, 0.1, 0.2\}$. The initial state is two Gaussian heat bumps and the boundary is held at zero.
3. Simulate u with RK4 and store 401 time steps.
4. Set the noise level $\sigma = \eta\times$ (RMS of the signal), with $\eta \in \{0, 1, 5, 10\%\}$.
5. Pick sensors from a fixed pool, so smaller sets sit inside larger ones ($5 \subset 20 \subset 50 \subset 100$). Layouts are random, grid, clustered or boundary-only.
6. Pick reading times from a fixed protocol (for example 8 uniform times, or only the final time).
7. Read the clean field at each sensor and time, and add Gaussian noise with a fixed seed.

Sweeps. Sensors $\{5, 20, 50, 100, \text{full}\}\times$ noise; horizon $\times$ time protocol; number of times $\times$ noise; layout $\times\{20, 50\}$ sensors. This gives 63 datasets per family, plus 11 variants: 389 in total.

Attributes and format. Each dataset has a sensor file with one row per reading and columns `x, y, t, u, sigma` (also the upload format of the app), a problem file (PDE, T, initial and boundary values), and the true D, which is used only for scoring. No pre-processing is applied; the PINN only rescales time by T.

Extra studies (same format): hypothesis suites H1–H9 (81 datasets with matched controls), a density extension (180 datasets, up to 3,969 sensors) and dense sets of 1,000–10,000 sensors.

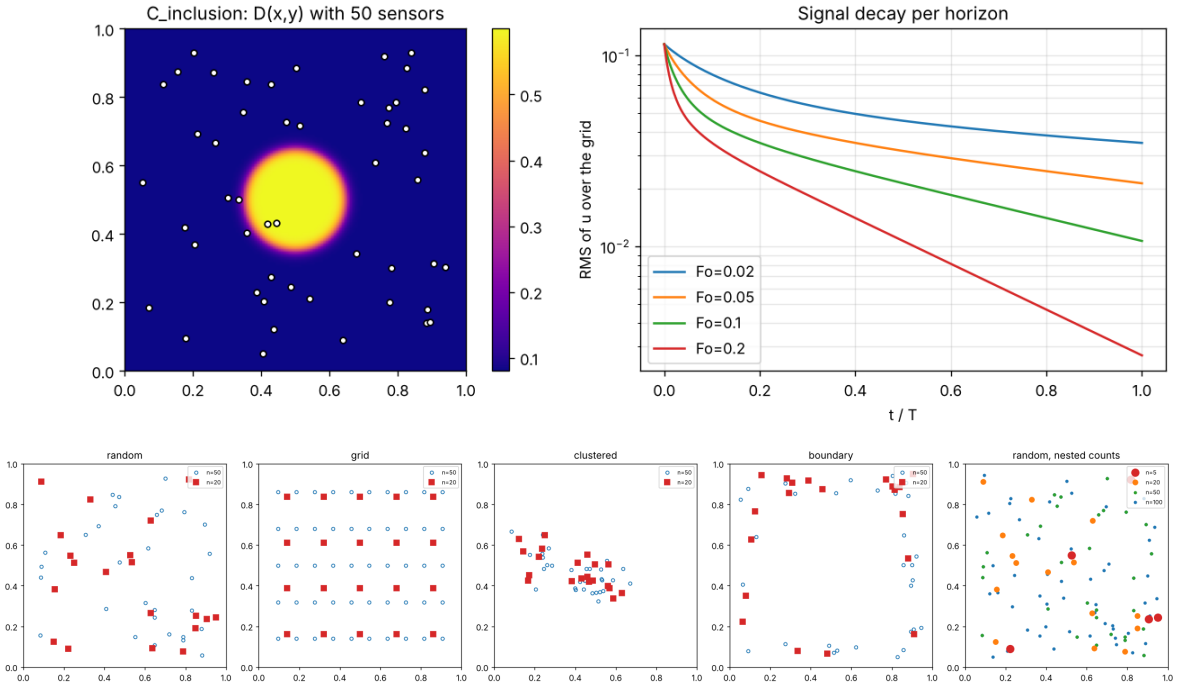


Figure 2: Top: the inclusion field with 50 random sensors, and how fast the signal decays for each horizon. Bottom: the sensor layouts and nested sensor counts.

6 Experiments

Metrics. (1) relL2(D)= $\|\hat{D} - D\|_2/\|D\|_2$ on the 65^2 grid; lower is better. (2) *Replay RMS/σ*: we run the estimated $\hat{D}$ through the Crank–Nicolson solver and compare with the readings. A value near 1 means the data are explained down to the noise level. It needs no true D, so this is the number shown to a user. (3) The best constant D: a method is only useful if it beats it.Used as a kind of a baseline

Training settings.

- *Classical:* L-BFGS-B, at most 150 iterations per α, α lowered by $\sqrt{10}$ each round until the discrepancy principle is met (Algorithm 1).
- *PINN:* AdamW, weight decay 10^{-5}, learning rates 1.5×10^{-3} (u) and 3.5×10^{-3} (D) with cosine decay to 10^{-5}, gradient clipping at 1.0, 1,500 epochs. We keep the best checkpoint from the last 35% of training. Algorithm 2 shows the procedure.
- *Dense-sensor study (both methods):* 1,000 Adam steps, then 10×400 quasi-Newton (SSBroyden) steps, as in Zhang et al.

Code and hardware. All code, trained models and results are on GitHub:
`https://github.com/bloodrepaer/IE643-Binary-Brains-PINN-Inversion`
We used PyTorch and Streamlit . The classical runs and the 20/44-dataset studies ran on CPUs (a laptop and 2-core cloud Linux machines, one thread per run). The neural studies also ran on a Windows laptop CPU, Kaggle notebooks (CPU, and GPU for some pilots) and Google Colab (T4 GPU).

The below experiments are all the experiments we have done till now:

1. **Verification:** solver order of accuracy, adjoint transpose identity, Taylor test and finite-difference gradient checks, and replaying the true D from the problem files.
2. **Classical inversion on all 389 datasets** (Tikhonov; TV also on the inclusion and defect families).
3. **PINN loss-weight sweep:** 10 settings on the Gaussian bump (50 sensors, 5% noise).
4. **20-dataset head-to-head** across all families, identical data and scoring.
5. **44-dataset sweep** on the inclusion and the bump: 5/20/50/100 random sensors $\times$ 0/1/5/10% noise, plus grid, clustered and boundary layouts.
6. **Hypothesis suites** (81 paired datasets): H1 very few sensors; H2 unknown boundary; H3 unknown heat source; H4 heavy noise; H5 long horizons; H6 more readings; H7 unknown boundary with more readings; H9 features finer than the control grid. Each test has a control that differs only in the tested factor. For unknown physics, both methods get the same assumed model (zero boundary, no source).
7. **Same network for D:** CN neural adjoint on all 389 datasets and the extra collections; neural PINN on a subset.
8. **Dense-sensor sweep:** 1,000/3,000/5,000/7,500 sensors $\times$ 1/5/10% noise on the smooth field (12 paired cells), plus a 10,000-sensor pair. A PINN win counts only if it has lower relL2(D) *and* passes a replay check ($\leq 1.1\sigma$ with both CN and BE) fixed before we looked at the truth.

7 Results

Solver and adjoint checks. Measured orders of accuracy are 4.03 (RK4), 2.01 (Crank–Nicolson) and 1.00 (forward Euler) (Figure 3). RK4 and forward Euler blow up just above their step-size limit, which is why inversion uses an implicit scheme. The adjoint passes every check: transpose identity to 4×10^{-16}, Taylor-test slopes 1.996–2.000, and agreement with finite differences to 10^{-8}. Replaying the true D gives median RMS/$\sigma = 1.01$ over 296 noisy datasets, so the problem files and the noise model are correct.

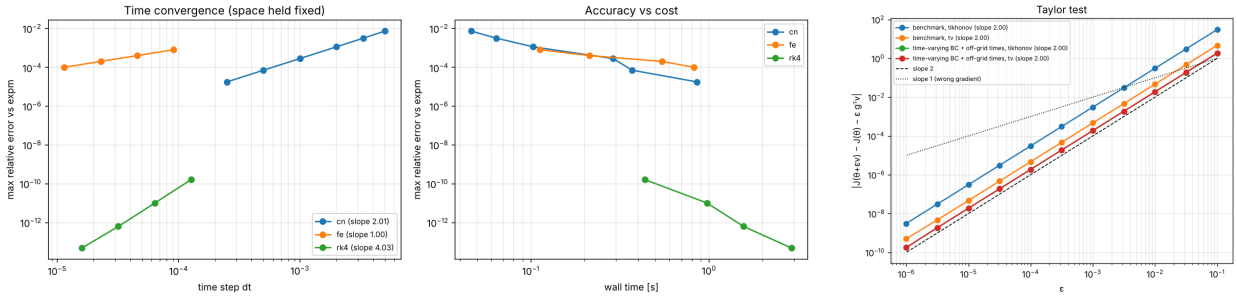


Figure 3: Left: convergence and cost of the three time steppers. Right: Taylor test of the adjoint gradient (slope 2).

Classical method on the benchmark. The adjoint fits 324 of the 389 datasets to the noise level (median replay 0.98σ). Its error drops with more sensors, more readings and less noise (Table 2). The inclusion is the hardest family: with 50 sensors it barely beats the best constant (0.66 vs 0.75). *Negative result:* on the layered field with $\leq 1\%$ noise, the smoothing weight becomes very small, the method fits solver error at the sharp layers, and the error goes up to 1.40.

	Sensors (5% noise)					Noise (50 sensors)			
Family	5	20	50	100	full	0%	1%	5%	10%
A smooth	0.319	0.128	0.103	0.110	0.077	0.060	0.075	0.103	0.135
B bump	0.394	0.289	0.229	0.189	0.075	0.014	0.119	0.229	0.317
C inclusion	0.690	0.631	0.661	0.462	0.215	0.354	0.439	0.661	0.713
D layered	0.211	0.319	0.216	0.240	0.149	1.404	0.975	0.216	0.300
E random field	0.327	0.186	0.207	0.175	0.069	0.056	0.116	0.207	0.253
F defects	0.236	0.193	0.182	0.160	0.112	0.113	0.118	0.182	0.201

Table 2: Classical adjoint, relL2(D) (Tikhonov, Fo 0.1, 8 uniform times, random sensors).

PINN tuning. Over 10 loss-weight settings on the bump, the best PINN error was 0.398, against 0.229 for the adjoint. *Negative result:* the PINN fitted the readings well ($\chi^2 \approx 0.95$) for nine quite different D maps (errors 0.40–0.65), but replaying those maps through the solver gave χ^2 from 3 to 56. A PINN can fit the data with the wrong coefficient. So we always score a result by replaying it through the solver, never by the network's own fit.

Head-to-head on 20 datasets. The adjoint has the lower error on all 20 datasets (median 0.23 vs 0.47, about 2× better). It beats the best constant on 20/20, the PINN on 8/20. Median replay is 0.97σ (adjoint) vs 3.7σ (PINN), and median CPU time is 46 s vs 125 s. The gap is smallest on the hardest cases (sharp inclusion, few sensors) and largest with rich data (full field: 0.030 vs 0.405). The recovered maps in Figure 4 show this qualitatively: the adjoint recovers the shape of the fields, while the PINN map is smoother and misses detail.

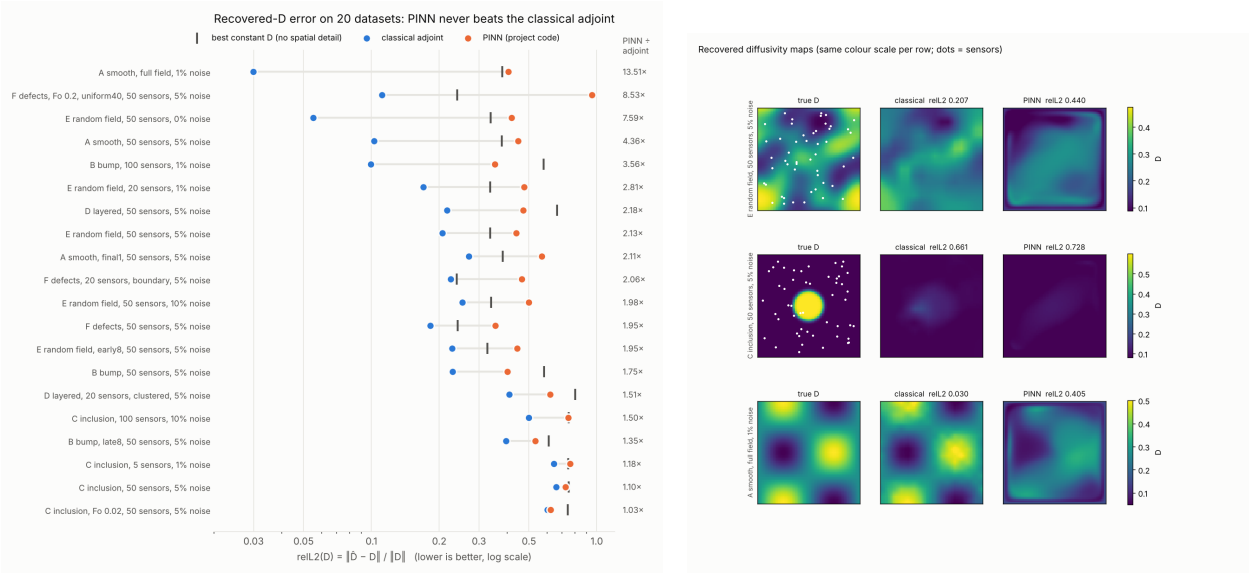


Figure 4: Left: relL2(D) on 20 datasets (log scale; bars are the best constant). Right: true and recovered maps for three of them.

Sensors, layout and noise (44 datasets). The adjoint has lower error on 41 of 44 (median 0.36 vs 0.61; best constant 0.68). On the bump both methods improve with more sensors and less noise, and the adjoint is 1.2–24× better. On the inclusion the PINN stays at 0.68–0.76, close to the best constant, at every setting. A grid layout beats random sensors, and boundary-only sensors are the worst. The PINN's three wins are all on the inclusion with too little information (5 random sensors; 20 and 50 boundary sen-

sors). There the PINN stays near the constant while the adjoint fits noise and invents structure.

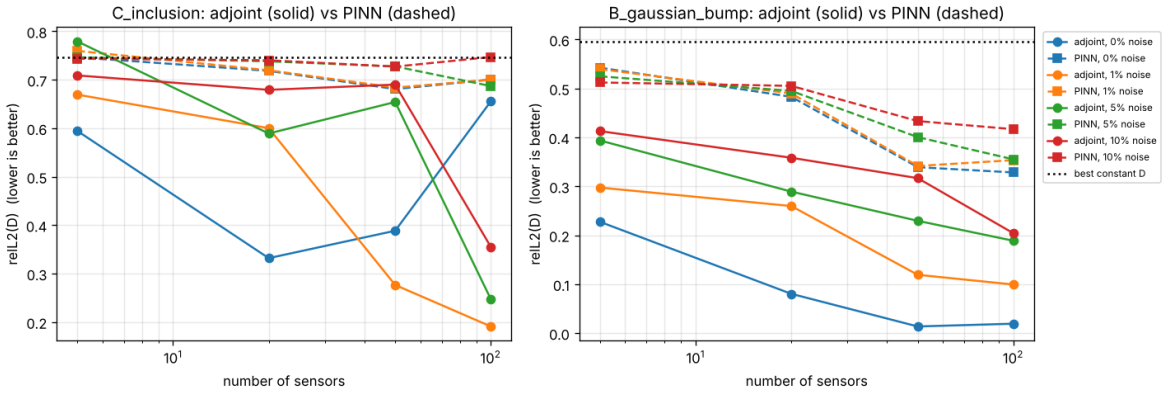


Figure 5: relL2(D) against number of sensors at four noise levels; adjoint solid, PINN dashed, best constant dotted.

Where the PINN is better: hypothesis suites. With the physics fully known, the adjoint wins 58 of 59 datasets (Table 3, Figure 6). With an *unknown boundary condition or source*, the PINN wins 21 of 22. Figure 7 shows why: once the assumed boundary is wrong, even the *true* D replays at about 9–21σ, so a method that enforces the PDE exactly must bend D to explain the data (median error 5.0). The PINN's soft penalty, and its free walls in H2/H7, absorb the mismatch (0.38, below the best constant 0.58). Here the adjoint is forced to assume a zero boundary and no source

Group	Datasets	PINN wins	Adjoint	PINN	Best const.
Physics known (H1, H4–H6, H9, controls)	59	1	0.123	0.412	0.344
BC or source unknown (H2u, H3u, H7u)	22	21	5.011	0.379	0.580

Table 3: Hypothesis suites: number of PINN wins and median relL2(D). "Unknown" means hidden from both methods.

Same network for D, and dense sensors. Giving the adjoint the same network for D did not change its accuracy: on the 389 datasets the CN neural adjoint and the grid adjoint have median error 0.226 vs 0.225, a tie. With up to a few hundred sensors the neural PINN behaves like the original PINN: it fits the sensors, but its D replays at about 5.8σ (median error 0.61).

With very dense data the picture changes, At 5% noise the PINN error falls from 35.4% (1,000 sensors) to 8.6% (7,500), while the BE neural adjoint stays at 21–25%. At 7,500 sensors and 5% noise both methods pass both replay checks; this is the only one of the 12 cells where a PINN win passes the checks. A 10,000-sensor pair gives 6.8% vs 26.4%. At 1% noise the PINN has lower error but fails the replay checks. The adjoint's error also rises slightly with density, which may come from its coarse backward-Euler time step.These are single-field, single-seed results, so they show a regime worth studying, not a general rule.

Interface. Figure 9 shows the results tab for the inclusion dataset. Classical: error 0.661, replay 0.98σ. PINN: 0.730, 2.38σ. Best constant: 0.751.

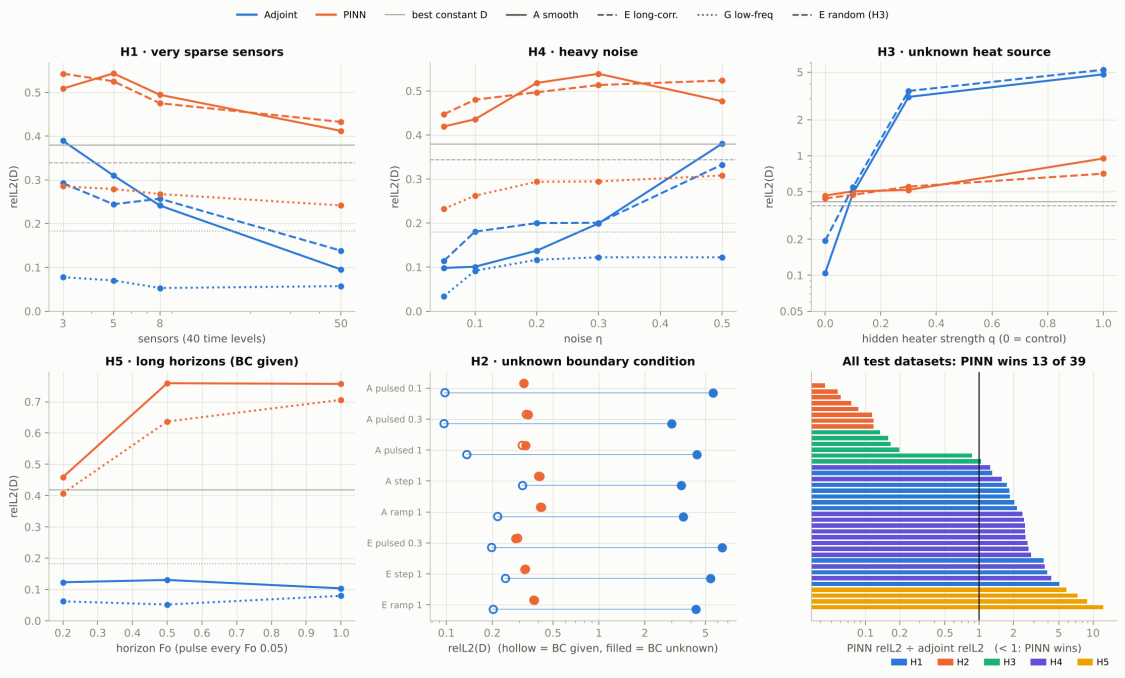


Figure 6: Hypothesis suites H1–H5. The PINN wins only where the boundary condition (H2) or heat source (H3) is unknown.

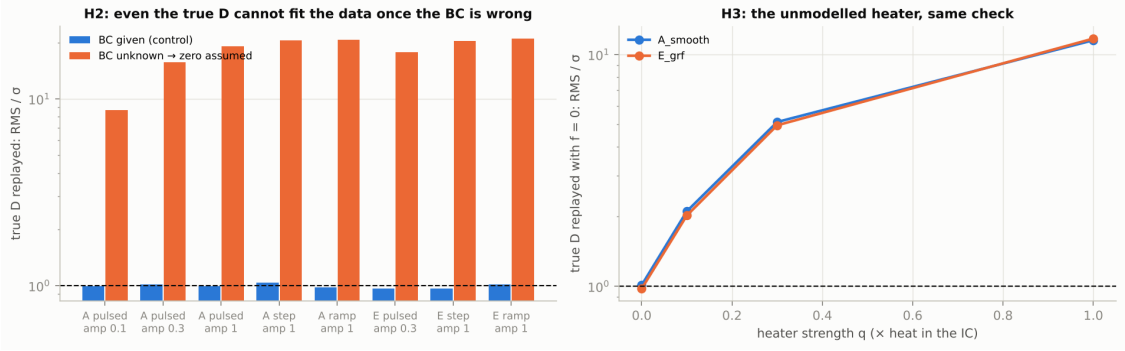


Figure 7: Why the adjoint fails under unknown physics: even the true D cannot fit the data under the assumed boundary (left) or source (right).

Sensors	Noise	PINN relL2(D)	Adjoint relL2(D)	PINN replay CN/BE (σ)	Adjoint replay CN/BE (σ)
1,000	5%	35.37%	21.45%	1.924 / 2.005	1.076 / 0.967
3,000	5%	16.48%	23.01%	1.136 / 1.150	1.095 / 0.992
5,000	5%	13.03%	23.79%	1.102 / 1.068	1.098 / 0.992
7,500	5%	**8.62%**	25.09%	1.097 / 1.025	1.098 / 0.994
7,500	1%	5.61%	45.64%	2.969 / 1.873	2.595 / 1.068
7,500	10%	22.64%	23.26%	1.098 / 1.122	1.019 / 0.993

Table 4: Dense-sensor sweep (smooth field, PINN vs BE neural adjoint). Replay check: $\leq 1.1\sigma$.

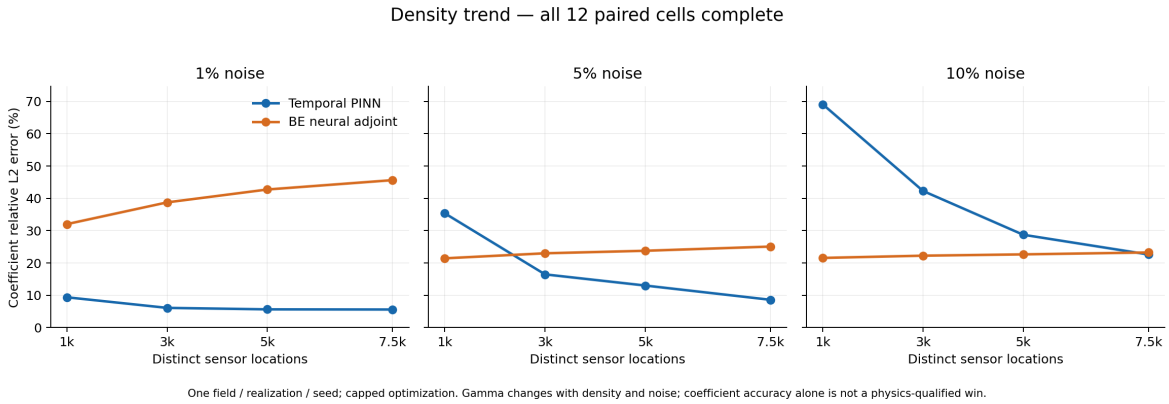


Figure 8: Coefficient error against sensor density at three noise levels (12 paired cells).

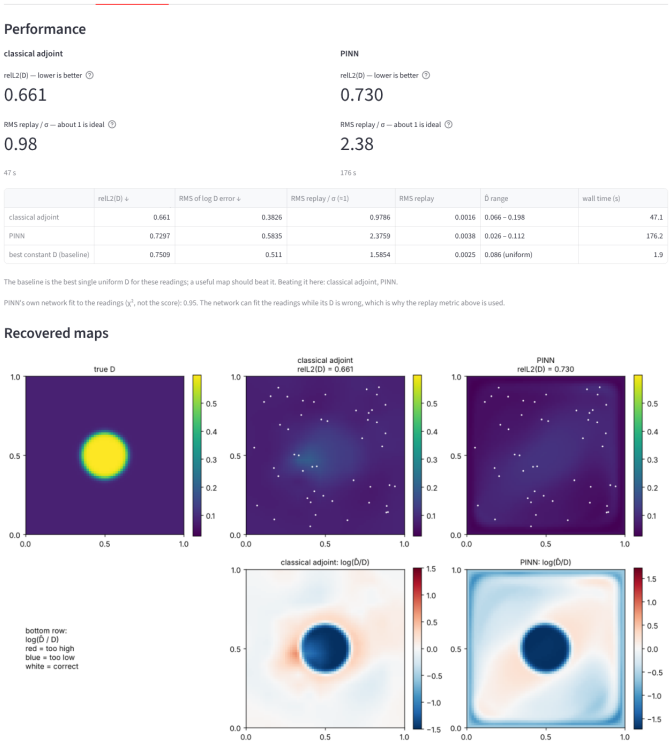


Figure 9: Interface results tab: metrics, recovered maps and $\log(\hat{D}/D)$ error maps.

8 Plan for Novelty Assessment

For the novelty assessment we will build a **hybrid PINN→adjoint model**. Our results show that the classical adjoint is the most accurate when the physics is known, but it always starts from a constant guess and can fail when data are sparse or the boundary is wrong. The PINN is less accurate but more robust in exactly these cases as we have seen so far. In the hybrid model we first train the PINN, then use its map of D as the starting point for the adjoint, which refines it with the exact solver. When the boundary condition is unknown, the PINN's estimate of the wall temperature can also be given to the adjoint. We will

compare the hybrid with each method alone on the same datasets, using the same error and replay metrics and counting the total cost including PINN training. The aim is to develop this one method that keeps the adjoint's accuracy and the PINN's robustness.

9 Conclusion

We studied how to recover a spatially varying diffusivity from sparse, noisy heat data. We built a verified solver, 389 benchmark datasets, a classical adjoint , a two-network PINN, a comparison between both these methods and a streamlit interface. When the physics is fully known and there are up to a few hundred sensors, the classical method is about twice as accurate, fits the data to the noise level and is faster. The PINN is the better choice when the boundary condition or a source is unknown, when the data are very dense, and when the data are too sparse, where it fails but not as much as Adjoint method. Its strength is flexibility and robustness to imperfect physics, not accuracy on a perfectly specified problem. Next we will combine both these methods and make a hybrid model in the novelty assessment that aims to keep PINN's robustness and Adjoint's accuracy

References

[1] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, Journal of Computational Physics, 2019. https://doi.org/10.1016/j.jcp.2018.10.045

[2] Sukirt Thakur, Ehsan Esmaili, Sarah Libring, Luis Solorio, and Arezoo M. Ardekani. Inverse resolution of spatially varying diffusion coefficient using physics-informed neural networks, Physics of Fluids, 2024. https://arxiv.org/abs/2403.03970

[3] Zhen Zhang, Alessandro Alla, and George Em Karniadakis. Adjoint method versus physics-informed neural networks in PDE-constrained inverse problems, arXiv preprint, 2026. https://arxiv.org/abs/2606.12337